

En kort introduktion til Maskinlæring; Pendul

STEMAcademyML

Januar 2026

I fysik vil vi ofte gerne bruge eksisterende data til at forudsige ting. Hvis vi har rigtig meget forskelligt data, kan det være svært for os som mennesker at overskue hvad sammenhængen er.

I dette modul vil vi prøve at give data fra et eksperiment i fysik til en machine learning algoritme, og se om den kan gennemskue hvilke parametre der er vigtige, og om den kan forudsige resultaterne.

Introduktion / teori / Pendul

Som fysikere, er vi interesserede i at finde ud af hvordan verden hænger sammen. Det gør vi ofte ved at prøve os frem med eksperimenter. Men forestil dig at du for første gang skal prøve at beskrive et pendul. Du kan se at det bevæger sig på en bestemt måde, og tænker at der nok kan skrives en ligning for det. Men hvordan i alverden skal den ligning se ud?

Du beslutter dig for at tage din computer med på råd. Den er nemlig ekstremt dygtig til at se sammenhænge i data, at kan hjælpe dig med at finde ud af hvilke parametre der er vigtige, og kan give dig en ide om hvad der skal inkluderes i din ligning.

0.1 Pendul

Kort om pendul og ligningen for pendul - eller skal det inkluderes i intro?

Rent teoretisk har vi en meget smuk og enkel ligning for svingningstiden (eller perioden) på et pendul.

$$T = 2\pi\sqrt{\frac{L}{g}} \quad (1)$$

Hvor g er tyngdeaccelerationen, og L er snorens længde. Vi ser at svingningstiden faktisk kun afhænger af snorens længde, resten er konstanter. Men, i udledningen af denne ligning bliver der lavet en del antagelser, blandt andet - snoren er masseløs - vinklen er lille (noget vi ofte gør i teoretiske udledninger kaldet small angle approximation) - luftmodstanden er ubetydelig

Til eftertanke: Hvorfor kan vi lave disse antagelser? Og holder de i den virkelige verden?

1 Modul 1: Neurale Netværk

Machine learning er et stærkt værktøj i videnskab, fordi den er ekspert i at se mønstre, især i situationer hvor mennesker har svært ved at se dem fordi der er store mængder data eller mange variabler. Hvis vi tager en masse data fra et forsøg, kan det være svært at se en sammenhæng hvis ikke vi kender teorien og har en ligning vi kan støtte os op af. Men en algoritme behøver ikke kender noget som helst til systemet. Den kan bare kigge på dataen, og se om der er nogle sammenhænge. (kom med et godt eksempel her?). Men hvordan virker det så?

Når man snakker machine learning, hører man ofte om **Neurale Netværk (NN)**. Navnet kommer fra, at opbygningen minder om måden neuroner i hjernen sender signaler på. Netværket består af **lag**, med **node**, og hver node har en **vægt**, der bestemmer, hvor meget et input betyder for resultatet.

Et simpelt neuralt netværk kan for eksempel se sådan ud:

Det består af et **input layer** med vores parameter, et **output layer** med resultatet, den forudsagte værdi, og ind imellem er der et antal **hidden layers**.

I hver node bliver der lavet en ligning med de input parameter der er forbundet til. Kigger vi for eksempel på den første node i det første hidden layer, ville der blive lavet en ligning der så således ud:

$$a \cdot \text{parameter1} + b \cdot \text{parameter2} + c \cdot \text{parameter3} + \text{bias} = \text{output}. \quad (2)$$

Den sender så outputtet videre som en parameter til det næste lag. for eksempel ville den første node i den andet hidden layer lave ligningen

$$a \cdot \text{output1} + b \cdot \text{output2} + c \cdot \text{output3} + d \cdot \text{output4} + \text{bias} = \text{output}. \quad (3)$$

og sådan fortsætter det igennem netværket indtil enden hvor den giver et endeligt tal.

Hvordan ved algoritmen hvilke værdier den skal give vægtene a , b , c , d og bias ? Der er her træning kommer ind i billedet. Computeren har nemlig ingen ide om hvilke værdier de skal have, så den prøver sig frem indtil den gætter rigtigt.

Det gør den ved først at prøve med tilfældige værdier for alle vægte og bias . Herefter kører den dataen igennem, og sammenligner de resultater den får, med de rigtige svar. Så justerer den de forskellige vægte lidt ad gangen, og ser om den forudsagte værdi kommer tættere på den sande værdi. For mennesker ville det her være en umulig opgave, alene i det "simple netværk" i figur xx har vi 35 parameter, forestil dig at skulle ændre på 35 forskellige værdier lidt ad gangen indtil du rammer rigtigt. Det ville tage dagevis at finde en god kombination.¹ Men med computerkraft kan vi gøre det på sekunder.

1.1 Hvordan ved algoritmen om den gør det rigtigt?

Vores algoritme har aldrig set ligningen for et pendul, så den har ingen ide om hvordan det i teorien fungerer, og kan ikke bare sætte tallene ind i ligningen og få svaret. Den skal i stedet kigge på dataen, og se om den kan finde mønstre, der kan hjælpe den til at komme med det rigtige svar. Men hvordan ved den, om den har fundet det rigtige svar? Det gør den ved det vi kalder **træning**. For at træne en algoritme, skal vi først lære den hvad der er rigtigt og forkert. Derfor skal vi give den eksempler på rigtige svar. I tilfældet her med pendulet, skal vi give den nogle eksempler på rigtige svar. Det ville være i form af en masse data hvor vi har målt svingningstiden.

1.2 Hvilke parameter er vigtige?

Når vi har trænet vores model, er det interessant at finde ud af, hvilke variable den lægger mest vægt på. Hvis vi skulle skrive en ligning for pendulet, ville det være godt først at vide hvilke parameter der betyder mest, og hvilke vi måske kan ignorere. En af de måder en algoritme vurderer hvilke parameter der er vigtige, kaldes **Permutation Importance**. Her tager man den trænede model og "roder" med én variabel ad gangen. Det vil sige, man blander dens værdier tilfældigt, så den mister sin sammenhæng med resten af dataen. Hvis modellens præcision falder meget, betyder det, at den variabel var vigtig for forudsigelsen. Hvis vi fx bytter tilfældigt rundt på vinklen, og modellens præcision falder markant, betyder det, at vinklen er en vigtig parameter. Omvendt, hvis det ikke gør den store forskel at bytte vinklen tilfældigt rundt, er den ikke vigtig for at kunne forudsige svingningstiden.

1.2.1 Hvordan vurderer vi hvor godt algoritmen klarer sig?

Når nu vi har fået opbygget vores neurale netværk, vil vi gerne finde ud af hvor god den er til at forudsige. Det gør vi ved at give modellen ny data, som den ikke har trænet på, og bede den om at forudsige, og så sammenligne med de rigtige værdier. Typisk deler man datasættet i to dele: en del til **træning**, som vi allerede har berørt, og en mindre del til **test**, som gemmes væk, indtil modellen skal vurderes. På den måde sikrer vi, at modellen ikke bare husker de præcise svar, men faktisk har lært et mønster, der også virker på ny data.

For regression vil man typisk visualisere performance som vist nedenfor: Til venstre er plottet forskellen mellem forudsagte og sande værdier (residualer). Her vil vi gerne have, at punkterne ligger tæt på nul, så forudsigelserne ikke systematisk rammer for højt eller for lavt, og at spredningen er så lav som muligt. Til højre er plottet forudsagte værdier mod sande værdier. Her vil vi gerne have, at punkterne ligger tæt på linjen $y = x$, som betyder, at forudsigelsen og den sande værdi er den samme.

1.3 Opgaver

1.3.1 Træn din algoritme

Nu hvor vi har en ide om hvordan en algoritme fungerer, skal vi prøve at træne vores model. For at træne en model, skal der bruges en stor mængde data, ofte taler vi 10.000 datapunkter og derved,

¹Til sammenligning har ChatGPT 1.8 trillioner parameter!

for at algoritmen kan finde meningsfulde mønstre. Heldigvis er der allerede sørget for et stort datasæt til formålet.

introducer dataen I datasættet er der målt ... Ved at sætte en algoritme til at lære af dataen, kan den fortælle os om den kan se andre mønstre end blot længden af snoren.

Hvilke parameter er vigtigst ifølge algoritmen? Stemmer det overens med hvad du ved om pendulet?

2 Modul 2: Dataindsamling

2.1 Indsaml data

(inkluder et ark (excel?) så de ved hvilke data de skal tage - der skal være de samme målinger som det data algoritmen har trænet på)

For at vi kan teste hvor godt vores algoritme klarer sig, skal vi bruge noget data. Derfor skal vi lave vores eget pendul forsøg.

Materialer: Stopur lod i forskellige størrelser og masse 2 meter fiskesnørre eller anden tynd snor 2 meter kæde eller anden type snor med vægt vinkelmåler et sted at sætte pendulet op

Opsætning:

data: skal det bare været et excel ark de kan skrive direkte i? om som kan uploades til siden bagefter?

Fremgangsmåde:

Find et sted hvor i kan hænge jeres pendul op, med minimum 2 meter til gulvet.

Til eftertanke: Hvilke kombinationer af forsøg vil den teoretiske ligning være god til at forudsige, og hvilke vil den have sværere ved? hvorfor?

3 Modul 3: Man vs Machine

Nu har vi en trænet machine learning algoritme, og en masse data fra vores forsøg. Nu skal vi teste om algoritmen kan klarer det ligeså godt som hvis vi regnede resultatet ud med ligninger - eller måske bedre?

(lav så de kan uploade deres data, og plotte hvor godt de klarer sig i forhold til algoritmen, residual plot og sand vs. forudsagte)

inkluder del hvor vi prøver udenfor fx vinklen som algoritmen har trænet på. Her kan vi se at den ikke klarer sig særlig godt på data den ikke har trænet på.

3.0.1 Spørgsmål

Hvad sker der hvis du prøver at forudsige svingsningstiden med en vinkel der er større end hvad algoritmen har trænet på? Er den stadig ligeså god?

3.1 Eftertanke...

Hvilke fordele og ulemper er der ved at bruge machine learning til at lære fysik?

Hvilke begrænsninger er der ved machine learning?

Hvilke begrænsninger er der ved ligningen? fx at den antager små vinkler.

Hvilken effekt har det om snoren er masseløs eller ej? (hint: massemidtpunkt)

4 (Ekstra) Modul 4: Gletjere

Hvordan bruges det så i den virkelige verden? Modsat eksemplet med penduler, så har vi mange situationer i den virkelige verden, hvor vi gerne vil kunne regne på, eller forudsige, om noget som vi ikke kender teorien bag eller en ligning på. Et eksempel i den virkelige verden fra klimafysik, er at forudsige størrelsen på en gletjser. I teorien lyder det måske nemt nok, størrelsen må jo bare være approksimativt længde*bredde*højde. Men i langt de fleste tilfælde kender vi ingen af de 3. Så vi mangler en ligning der kan forudsige størrelsen af en gletjer ud fra de ting vi rent fysisk kan måle, hvilket er størrelser som x , x og x . Og da vi ikke har en ligning for det, tager vi en algoritme til hjælp.

Men som i tilfældet med pendulet, skal vi bruge noget data at træne på. Til det har der været lavet målinger hvor man har?

5 Ekstramateriale - elever

Måske ekstramateriale skal være et kompendie for sig selv?

5.1 Hvordan fungerer algoritmen egentlig?

Til dem der er nysgerrige, kommer der her en lidt dybere og mere matematisk forklaring på algoritmen, inkluderet forklaring på activation function m.m.

6 Ekstramateriale - undervisere

En lille opsamling på teorien bag pendul?

Hjælp til hvordan de skal dele op i lab